

Qualité de code en Java

Bonnes pratiques, robustesse, tests et DevSecOps

Document pédagogique complet

7 avril 2026

Table des matières

1	Introduction	4
1.1	Objectif du document	4
1.2	Criticité	4
1.3	Contenu du document	4
1.4	Approche pédagogique	4
2	Conventions de nommage et casse	5
2.1	Mauvais code	5
2.2	Bon code	5
2.3	Explication	5
3	Constantes et enums	6
3.1	Mauvais code	6
3.2	Bon code	6
3.3	Explication	6
4	Commentaires utiles	7
4.1	Mauvais code	7
4.2	Bon code	7
4.3	Explication	7
5	Commentaires orientés intention	8
5.1	Mauvais code	8
5.2	Bon code	8
5.3	Explication	8
6	Javadoc et contrat d'API	9
6.1	Mauvais code	9
6.2	Bon code	9
6.3	Explication	9
7	Compilation stricte	10
7.1	Mauvais code	10
7.2	Bon code	10
7.3	Explication	10
8	NullPointerException	11
8.1	Mauvais code	11
8.2	Bon code	11
8.3	Explication	11

9	Gestion des retours null avec Optional	12
9.1	Mauvais code	12
9.2	Bon code	12
9.3	Explication	12
10	Comparaison d'objets	13
10.1	Mauvais code	13
10.2	Bon code	13
10.3	Explication	13
11	equals() et hashCode()	14
11.1	Mauvais code	14
11.2	Bon code	14
11.3	Explication	14
12	Immuabilité et records	15
12.1	Mauvais code	15
12.2	Bon code	15
12.3	Explication	15
13	Ressources non fermées	16
13.1	Mauvais code	16
13.2	Bon code	16
13.3	Explication	16
14	Sortie standard et journalisation	17
14.1	Mauvais code	17
14.2	Bon code	17
14.3	Explication	17
15	Exceptions avalées	18
15.1	Mauvais code	18
15.2	Bon code	18
15.3	Explication	18
16	Concurrence : visibilité mémoire	19
16.1	Mauvais code	19
16.2	Bon code	19
16.3	Explication	19
17	Concurrence : opération non atomique	20
17.1	Mauvais code	20
17.2	Bon code	20
17.3	Explication	20
18	Modification de collection pendant itération	21
18.1	Mauvais code	21
18.2	Bon code	21
18.3	Explication	21

19 API Stream	22
19.1 Mauvais code	22
19.2 Bon code	22
19.3 Explication	22
20 Performance : String en boucle	23
20.1 Mauvais code	23
20.2 Bon code	23
20.3 Explication	23
21 Tests unitaires	24
21.1 Mauvais code	24
21.2 Bon code	24
21.3 Explication	24
22 Couverture de code	25
22.1 Mauvais code	25
22.2 Bon code	25
22.3 Explication	25
23 Complexité excessive	26
23.1 Mauvais code	26
23.2 Bon code	26
23.3 Explication	26
24 Analyse de style avec Checkstyle	27
24.1 Mauvais code	27
24.2 Bon code	27
24.3 Explication	27
25 Analyse de bugs avec SpotBugs et PMD	28
25.1 Mauvais code	28
25.2 Bon code	28
25.3 Explication	28
26 Sécurité des dépendances	29
26.1 Mauvais code	29
26.2 Bon code	29
26.3 Explication	29
27 Inspection continue avec SonarQube	30
27.1 Mauvais code	30
27.2 Bon code	30
27.3 Explication	30
28 Conclusion	31

Chapitre 1

Introduction

1.1 Objectif du document

Ce document a pour objectif de présenter les bonnes pratiques essentielles en langage Java à travers une approche comparative entre **mauvais** et **bon code**. Il vise à améliorer la qualité, la robustesse et la maintenabilité des applications en mettant en évidence les erreurs courantes et leurs corrections.

1.2 Criticité

Java offre un cadre plus sûr que des langages bas niveau, mais de mauvaises pratiques peuvent malgré tout conduire à :

- des erreurs d'exécution comme les `NullPointerException`
- des fuites de ressources
- une dette technique importante

Adopter des règles strictes permet de produire un code lisible, fiable, testable et adapté à un contexte industriel.

1.3 Contenu du document

Ce document couvre les aspects fondamentaux de la qualité en Java :

- Respect des conventions de nommage et de structure
- Gestion correcte de `null`, des exceptions et des ressources
- Bon usage des objets, collections et API modernes
- Écriture de code lisible, documenté et maintenable

1.4 Approche pédagogique

Chaque chapitre suit une structure simple et efficace :

- un exemple de **mauvais code**
- une version corrigée (**bon code**)
- une **explication** claire du problème et de la solution

Cette approche permet de comprendre rapidement les erreurs fréquentes et d'adopter les bons réflexes en situation réelle.

Chapitre 2

Conventions de nommage et casse

2.1 Mauvais code

```
package Com.MyProject.Utills;

class usermanager {
    int User_Age;
    void GetData() {}
}
```

2.2 Bon code

```
package com.myproject.utils;

class UserManager {
    int userAge;

    void getData() {
    }
}
```

2.3 Explication

Les conventions Java standard imposent :

- les packages en minuscules;
- les classes en PascalCase;
- les méthodes et variables en camelCase.

Ces règles rendent le code immédiatement lisible et cohérent entre développeurs. Elles sont aussi vérifiées automatiquement par des outils comme Checkstyle.

Chapitre 3

Constantes et enums

3.1 Mauvais code

```
final int max = 7000;

enum state {
    on, off
}
```

3.2 Bon code

```
static final int MAX_RPM = 7000;

enum State {
    ON,
    OFF
}
```

3.3 Explication

Les constantes doivent être facilement identifiables. Une convention claire améliore la lisibilité et réduit les ambiguïtés entre variables ordinaires et invariants.

Chapitre 4

Commentaires utiles

4.1 Mauvais code

```
int i = 0; // initialise i a zero
```

4.2 Bon code

```
int index = 0;
```

4.3 Explication

Un commentaire ne doit pas répéter ce que le code exprime déjà clairement. Il doit expliquer pourquoi le code existe, quelle contrainte il respecte, ou quelle hypothèse il documente.

Chapitre 5

Commentaires orientés intention

5.1 Mauvais code

```
// wait a bit  
Thread.sleep(200);
```

5.2 Bon code

```
// Le capteur necessite 200 ms pour se stabiliser apres alimentation  
Thread.sleep(200);
```

5.3 Explication

Un commentaire utile explique l'intention métier ou technique. Il doit apporter une information que le code seul ne permet pas de déduire facilement.

Chapitre 6

Javadoc et contrat d'API

6.1 Mauvais code

```
/** get value */  
int getValue() {  
    return value;  
}
```

6.2 Bon code

```
/**  
 * Retourne la tension mesuree par le capteur.  
 *  
 * @return tension en millivolts (>= 0)  
 */  
int getValue() {  
    return value;  
}
```

6.3 Explication

La Javadoc définit le contrat d'une méthode : unités, hypothèses, valeurs de retour, exceptions éventuelles. Elle est essentielle pour les API publiques ou partagées.

Chapitre 7

Compilation stricte

7.1 Mauvais code

```
javac Main.java
```

7.2 Bon code

```
javac -Xlint:all -Werror Main.java
```

7.3 Explication

Compiler avec des warnings activés permet de détecter plus tôt des problèmes de qualité. Traiter les warnings comme des erreurs évite qu'ils s'accumulent dans le projet.

Chapitre 8

NullPointerException

8.1 Mauvais code

```
String role = getRole();  
if (role.equals("ADMIN")) {  
    grantAccess();  
}
```

8.2 Bon code

```
String role = getRole();  
if ("ADMIN".equals(role)) {  
    grantAccess();  
}
```

8.3 Explication

Appeler une méthode sur une référence potentiellement nulle provoque une `NullPointerException`. Placer le littéral à gauche évite ce risque.

Chapitre 9

Gestion des retours null avec Optional

9.1 Mauvais code

```
public User findUser(String id) {  
    if (db.notContains(id)) {  
        return null;  
    }  
    return db.get(id);  
}
```

9.2 Bon code

```
import java.util.Optional;  
  
public Optional<User> findUser(String id) {  
    if (db.notContains(id)) {  
        return Optional.empty();  
    }  
    return Optional.of(db.get(id));  
}
```

9.3 Explication

Retourner `null` oblige l'appelant à faire des vérifications manuelles, parfois oubliées. `Optional` rend explicite la possibilité d'absence de valeur.

Chapitre 10

Comparaison d'objets

10.1 Mauvais code

```
String a = new String("x");  
String b = new String("x");  
  
if (a == b) {  
}
```

10.2 Bon code

```
if (a.equals(b)) {  
}
```

10.3 Explication

L'opérateur `==` compare les références. Pour comparer le contenu de deux objets, il faut utiliser `equals()`.

Chapitre 11

equals() et hashCode()

11.1 Mauvais code

```
class User {
    String email;

    public boolean equals(Object o) {
        return (o instanceof User u)
            && email.equals(u.email);
    }
}
```

11.2 Bon code

```
class User {
    String email;

    public boolean equals(Object o) {
        return (o instanceof User u)
            && email.equals(u.email);
    }

    public int hashCode() {
        return email.hashCode();
    }
}
```

11.3 Explication

Si `equals()` est redéfini, `hashCode()` doit l'être aussi. Sinon les collections de hachage comme `HashMap` ou `HashSet` se comportent mal.

Chapitre 12

Immuabilité et records

12.1 Mauvais code

```
public class Point {  
    private final int x;  
    private final int y;  
  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public int getX() { return x; }  
    public int getY() { return y; }  
}
```

12.2 Bon code

```
public record Point(int x, int y) {}
```

12.3 Explication

Les **record** réduisent fortement le code répétitif. Ils fournissent automatiquement un constructeur, des accesseurs, `equals()`, `hashCode()` et `toString()`.

Chapitre 13

Ressources non fermées

13.1 Mauvais code

```
FileInputStream in = new FileInputStream("data.txt");  
byte[] data = in.readAllBytes();  
in.close();
```

13.2 Bon code

```
try (FileInputStream in = new FileInputStream("data.txt")) {  
    byte[] data = in.readAllBytes();  
}
```

13.3 Explication

Si une exception survient, une ressource ouverte manuellement peut ne jamais être libérée. Le `try-with-resources` garantit la fermeture correcte.

Chapitre 14

Sortie standard et journalisation

14.1 Mauvais code

```
try {
    processData();
    System.out.println("Traitement termine");
} catch (Exception e) {
    e.printStackTrace();
}
```

14.2 Bon code

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

private static final Logger LOGGER =
    LoggerFactory.getLogger(MyClass.class);

try {
    processData();
    LOGGER.info("Traitement termine avec succes");
} catch (Exception e) {
    LOGGER.error("Erreur lors du traitement des donnees", e);
}
```

14.3 Explication

`System.out.println()` et `printStackTrace()` sont peu adaptés à un code industriel. Une bibliothèque de logging permet un meilleur contrôle, un meilleur formatage et une meilleure intégration aux outils de supervision.

Chapitre 15

Exceptions avalées

15.1 Mauvais code

```
try {  
    save();  
} catch (Exception e) {  
}
```

15.2 Bon code

```
try {  
    save();  
} catch (IOException e) {  
    LOGGER.error("Erreur sauvegarde", e);  
    throw new CustomException("Impossible de sauvegarder", e);  
}
```

15.3 Explication

Un `catch` vide masque complètement l'erreur. Il faut au minimum journaliser le problème, puis le propager ou le transformer en exception métier.

Chapitre 16

Concurrence : visibilité mémoire

16.1 Mauvais code

```
boolean stop = false;

void loop() {
    while (!stop) {}
}
```

16.2 Bon code

```
volatile boolean stop = false;

void loop() {
    while (!stop) {}
}
```

16.3 Explication

Sans `volatile` ou synchronisation, un thread peut ne jamais observer la mise à jour faite par un autre thread. Cela peut provoquer des boucles infinies.

Chapitre 17

Concurrence : opération non atomique

17.1 Mauvais code

```
int count = 0;

void inc() {
    count++;
}
```

17.2 Bon code

```
import java.util.concurrent.atomic.AtomicInteger;

AtomicInteger count = new AtomicInteger(0);

void inc() {
    count.incrementAndGet();
}
```

17.3 Explication

L'opération ++ n'est pas atomique. En environnement multithread, il faut utiliser des primitives adaptées comme `AtomicInteger`.

Chapitre 18

Modification de collection pendant itération

18.1 Mauvais code

```
for (String s : list) {  
    if (s.isBlank()) {  
        list.remove(s);  
    }  
}
```

18.2 Bon code

```
list.removeIf(String::isBlank);
```

18.3 Explication

Modifier une collection pendant une boucle `for-each` provoque souvent une `ConcurrentModificationException`. Il faut utiliser une méthode adaptée.

Chapitre 19

API Stream

19.1 Mauvais code

```
List<String> activeNames = new ArrayList<>();
for (User user : users) {
    if (user.isActive()) {
        activeNames.add(user.getName().toUpperCase());
    }
}
```

19.2 Bon code

```
List<String> activeNames = users.stream()
    .filter(User::isActive)
    .map(user -> user.getName().toUpperCase())
    .toList();
```

19.3 Explication

L'API Stream permet d'exprimer plus directement l'intention : filtrer, transformer, collecter. Elle réduit souvent le bruit technique des boucles manuelles.

Chapitre 20

Performance : String en boucle

20.1 Mauvais code

```
String s = "";  
for (int i = 0; i < 10000; i++) {  
    s += i;  
}
```

20.2 Bon code

```
StringBuilder sb = new StringBuilder();  
for (int i = 0; i < 10000; i++) {  
    sb.append(i);  
}  
String s = sb.toString();
```

20.3 Explication

Les objets `String` sont immuables. Les concaténations répétées créent beaucoup d'objets temporaires; `StringBuilder` est plus efficace.

Chapitre 21

Tests unitaires

21.1 Mauvais code

```
// Aucun test associe
int add(int a, int b) {
    return a + b;
}
```

21.2 Bon code

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

class CalculatorTest {
    @Test
    void shouldAddTwoNumbers() {
        assertEquals(4, add(2, 2));
    }
}
```

21.3 Explication

Les tests unitaires permettent de valider le comportement réel du code et de détecter les régressions. Ils sont indispensables dans une chaîne de qualité moderne.

Chapitre 22

Couverture de code

22.1 Mauvais code

```
// Code de production jamais execute pendant les tests
```

22.2 Bon code

```
mvn jacoco:report
```

22.3 Explication

La couverture mesure la part du code réellement exécutée pendant les tests. Elle permet d'identifier des zones non testées.

Chapitre 23

Complexité excessive

23.1 Mauvais code

```
if (a) {  
    if (b) {  
        if (c) {  
            if (d) {  
                work();  
            }  
        }  
    }  
}
```

23.2 Bon code

```
if (a && b && c && d) {  
    work();  
}
```

23.3 Explication

Une trop forte imbrication rend le code difficile à lire, à tester et à maintenir. Réduire la complexité améliore la qualité globale du logiciel.

Chapitre 24

Analyse de style avec Checkstyle

24.1 Mauvais code

```
// Aucun controle automatique du style
```

24.2 Bon code

```
mvn checkstyle:check
```

24.3 Explication

Checkstyle vérifie automatiquement le respect des conventions de style et de formatage. Cela homogénéise le code à l'échelle du projet.

Chapitre 25

Analyse de bugs avec SpotBugs et PMD

25.1 Mauvais code

```
// Aucun scan de bugs ou mauvaises pratiques
```

25.2 Bon code

```
mvn spotbugs:check  
mvn pmd:check
```

25.3 Explication

Ces outils détectent des défauts fréquents, des mauvaises pratiques et certains problèmes de robustesse ou de sécurité. Ils complètent utilement les tests.

Chapitre 26

Sécurité des dépendances

26.1 Mauvais code

```
// Dépendances tierces jamais vérifiées
```

26.2 Bon code

```
mvn dependency-check:check
```

26.3 Explication

Une part importante des vulnérabilités vient des bibliothèques tierces. Il faut analyser régulièrement les dépendances utilisées.

Chapitre 27

Inspection continue avec SonarQube

27.1 Mauvais code

```
// Aucun quality gate centralise
```

27.2 Bon code

```
mvn clean verify sonar:sonar \  
-Dsonar.projectKey=mon-projet-java \  
-Dsonar.host.url=http://localhost:9000 \  
-Dsonar.token=mon_token_secret
```

27.3 Explication

SonarQube centralise la dette technique, les bugs, les vulnérabilités et la couverture. Cela permet de mettre en place une vraie quality gate dans l'intégration continue.

Chapitre 28

Conclusion

Un code Java de qualité professionnelle en approche DevSecOps repose sur :

- des conventions de nommage et de style claires ;
- des API bien documentées ;
- une gestion maîtrisée de `null`, des exceptions et des ressources ;
- une attention particulière à la concurrence et à la lisibilité ;
- des tests unitaires et une couverture mesurée ;
- des analyses automatiques de style, de bugs, de sécurité et de dette technique.

Ces pratiques permettent de produire un code plus robuste, plus maintenable, plus sûr et plus industrialisable.